InfraQ

A Scheduling Gateway for Shared Local LLM Inference

Advanced Cloud Programming • CW3 • University of Edinburgh • April 2026

InfraQ implements four LLM request scheduling strategies on top of RabbitMQ, Redis, PostgreSQL, and a Python asyncio worker, running on Mac M1 without CUDA. A built-in benchmark framework measures each strategy under controlled workloads. Across 45 runs of 100-request workloads, cache-aware dispatching on repeated prompts achieved ***5.6× higher throughput*** *and* ***76% lower average latency*** *compared to non-cached strategies.*

Problem and Idea

Teams running a shared Ollama instance face three problems: concurrent requests pile up with no queue, repeated prompts get re-inferred every time, and there is no way to measure whether a different scheduling approach would help. vLLM and SGLang solve this at the GPU kernel level, but both require NVIDIA hardware most small teams cannot justify. LiteLLM proxies requests; it does not schedule them.

InfraQ moves continuous batching[1] and prefix caching[2] from the GPU layer to the infrastructure layer, approximated with RabbitMQ slot management and a Redis text cache. More importantly, InfraQ is a **research platform**: switching strategies requires changing one Redis key, and a benchmark framework controls the full experiment lifecycle—idle confirmation, strategy application, cache clearing, request submission—producing results comparable across strategies on identical workloads.

Scheduling Strategies

Sequential processes one request at a time. No concurrency. It is the performance floor.

Static batching collects N requests, fires them all at once, then waits for the entire batch before accepting more. The slowest request holds every completed slot idle—a structural source of tail latency.

Continuous dispatching (after vLLM) maintains K concurrent slots via an asyncio semaphore. Any slot that completes immediately admits the next request, eliminating batch-boundary stalls. Ollama receives K simultaneous HTTP connections and, with `OLLAMA_NUM_PARALLEL` set accordingly, merges them into a single tensor forward pass—InfraQ manages the slot lifecycle; Ollama handles batched inference.

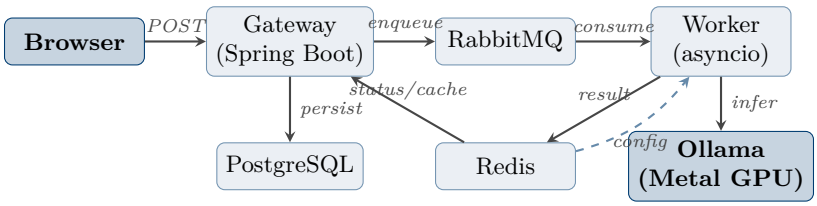
Cache-aware dispatching (after SGLang) adds a Redis lookup before any slot is acquired. The cache key is the SHA-256 hash of model name and prompt text. Hits return in ∼11 ms without touching Ollama; misses proceed through continuous dispatching and write their result to Redis (TTL: 1 hour). This approximates KV-cache reuse at the infrastructure layer without shared GPU memory.

The worker writes its runtime state to Redis every second. When the desired configuration changes, the worker drains all in-flight tasks and restarts under the new strategy without container restarts, enabling clean back-to-back experiments.

Architecture

The gateway (Spring Boot) returns 202 immediately on every request, writing to PostgreSQL, Redis, and RabbitMQ in sequence. The worker (Python asyncio) consumes from RabbitMQ and applies the active scheduling strategy. Ollama runs natively on the host, accessed via `host.docker.internal`—Mac Metal GPU acceleration requires no passthrough.

[1] Kwon et al., *Efficient Memory Management for LLM Serving with PagedAttention*, SOSP 2023.
[2] Zheng et al., *SGLang: Efficient Execution of Structured Language Model Programs*, NeurIPS 2024.



Redis serves four roles: request status fast-path for browser polling (<1 ms); prompt text cache; real-time atomic metric counters; and the configuration channel between gateway and worker (dashed). PostgreSQL stores permanent records—request histories, per-request timing, and benchmark run statistics that outlast Redis TTLs.

Experimental Results

All experiments: MacBook Air M1 8 GB, Qwen2.5:1.5b via Ollama, 100 requests per run, 5 repeats per condition. Data: `summary.csv` and `manifest.json` in submission.

Table 1: Experiment A — unique prompts (no cache benefit). 5-run averages.

Strategy	Slots	Avg latency	P95	Throughput
Sequential	1	188 s	442 s	0.213 req/s
Static batching	8	185 s	438 s	0.213 req/s
Continuous	8	180 s	**426 s**	**0.219 req/s**
Cached	8	179 s	430 s	0.218 req/s

Table 2: Experiment B — repeated prompts (hot prompt set). 5-run averages.

Strategy	Slots	Avg latency	P95	Throughput	Hit rate
Sequential	1	311 s	584 s	0.165 req/s	0%
Static batching	4	298 s	557 s	0.173 req/s	0%
Continuous	4	299 s	565 s	0.170 req/s	0%
Cached	**4**	**73 s**	**96 s**	**0.961 req/s**	**71%**

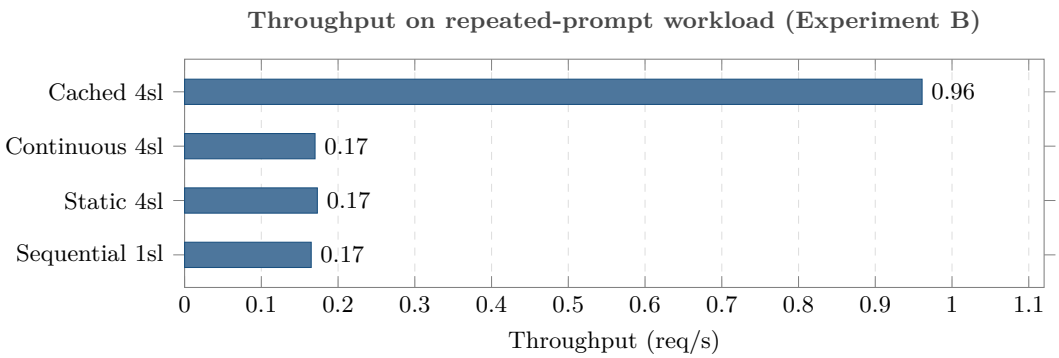


Figure 1: Cache-aware dispatching (71% hit rate) achieves 5.6× higher throughput on repeated prompts. Non-cached strategies are bound by Ollama's total token generation rate.

On **unique prompts**, scheduling gains are modest. Continuous dispatching reduces P95 latency by 12 s versus static batching (426 s vs. 438 s), consistent with eliminating the wait-for-slowest-in-batch effect. Total token generation rate is the bottleneck; concurrency alone cannot exceed it.

On **repeated prompts**, caching is the dominant lever. At 71% hit rate, cache-aware dispatching delivers 5.6× higher throughput and 76% lower average latency (73 s vs. 299 s). Cache hits bypass Ollama entirely; P95 falls from ∼565 s to 96 s.

Slot-count ablation (cached strategy, repeated workload): 4 slots (0.961 req/s, 71% hit rate) outperforms 8 slots (0.699 req/s, 67% hit rate). More slots produce more simultaneous cache misses before warmup, reducing hit rate and increasing Ollama contention simultaneously. The optimal slot count for cache-aware dispatching is not "as many as possible."

Completeness

All four strategies are fully implemented and hot-switchable on a running system. The benchmark framework runs automatically across a 45-run matrix (100 requests per run, 5 repeats per condition). A web interface provides a chat tab, a real-time metrics dashboard, and benchmark history with per-run comparison charts. The system starts with `docker compose up`; Ollama and the Qwen2.5:1.5b model are the only external dependencies.

AI Usage

Claude Code (Anthropic) assisted with Spring Boot boilerplate, Python asyncio control flow, and system design discussion. Strategy definitions, experimental methodology, and result analysis are the author's own work. All AI-generated code was tested and debugged locally before submission.